

Database System

Instructor: Yiwen Shen, Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr

Relational Databases

Introduction

- **Structure of Relational Databases**
- **Database Schema**
- **Keys**
- **Schema Diagrams**
- **Relational Query Languages**
- **The Relational Algebra**

Example of an Instructor Relation

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

attributes
(or columns)

tuples
(or rows)

Relation Schema and Instance

- $A_1, A_2, \dots, A_n$ are *attributes*
- $R = (A_1, A_2, \dots, A_n)$ is a *relation schema*

Example:

instructor = (*ID*, *name*, *dept_name*, *salary*)

- A relation *instance* r defined over *schema* R is denoted by $r(R)$.
- The current values a relation are specified by a *table*
- An element t of relation r is called a *tuple* and is represented by a *row* in a table

Attributes

- The **set of allowed values** for each attribute is called the **domain** of the attribute
- **Attribute values** are (normally) required to be **atomic**; that is, **indivisible**
- The special value **null** is a member of every domain. Indicated that the value is “unknown”
- The null value causes complications in the definition of many operations

Relations are **Unordered**

- Order of tuples is irrelevant (tuples may be stored in an arbitrary order)
 - Example: *instructor* relation with unordered tuples

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Database Schema

数据库模式

- **Database schema** -- is the **logical structure** of the database.
- **Database instance** -- is a **snapshot** of the data in the database at a given instant in time.
- **Example:**
 - **schema:** instructor (ID, name, dept_name, salary)
 - **Instance:**

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Keys

- Let $K \subseteq R$

超级键 • K is a **superkey** of R if values for K are sufficient to *identify a unique tuple* of each possible relation $r(R)$

- Example: $\{ID\}$ and $\{ID, name\}$ are both superkeys of instructor.

- Superkey K is a **candidate key** if K is **minimal**

- Example: $\{ID\}$ is a candidate key for Instructor

- One of the candidate keys is selected to be the **primary key**. It is **underlined**.

主键

- Which one?

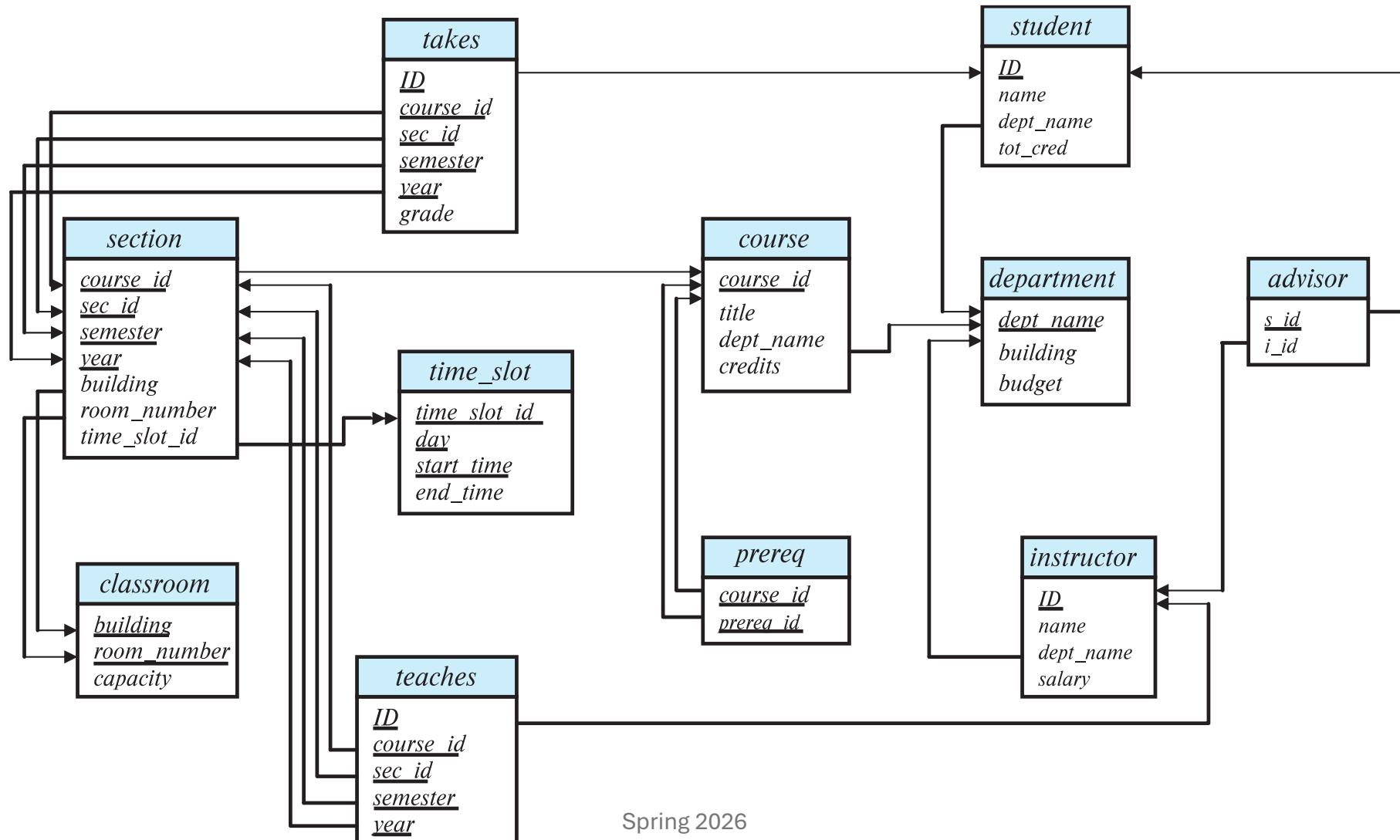
外键 • **Foreign key** constraint: Value in one relation must appear in another

- **Referencing** relation: the relation has the primary key.

- **Referenced** relation: the relation has the primary key of another relation.

- Example: *dept_name* in *instructor* is a foreign key from *instructor* referencing *department*

Schema Diagram for University Database



Relational Query Languages

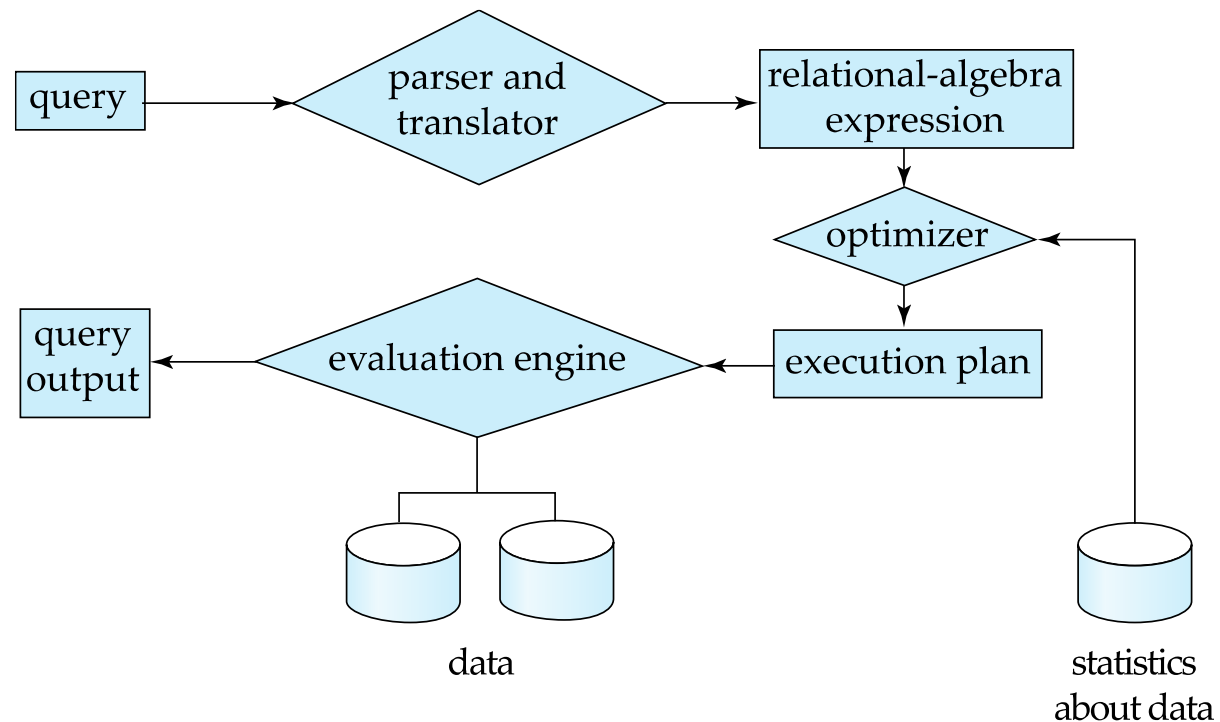
- Procedural versus non-procedural, or declarative
- “Pure” languages:
 - Relational algebra
 - Tuple relational calculus
 - Domain relational calculus
- The above 3 pure languages are equivalent in computing power
- We will concentrate in this lecture on **relational algebra**
 - Not Turing-machine equivalent
 - Consists of **6 basic operations**

Relational Algebra 关系代数

- A procedural language consisting of a set of operations that take one or two relations as input and produce a new relation as their result.
- Six basic operators
 - **select**: σ (Sigma)
 - **project**: π (Pi)
 - **union**: $\cup$ (**Intersection**: $\cap$)
 - **set difference**: $-$
 - **Cartesian product**: $\times$
 - **rename**: ρ (Rho)

Query Processing

- 1. Parsing and translation
- 2. Optimization
- 3. Evaluation



Select Operation

- The **select** operation selects tuples that satisfy a given predicate.
 - Notation: $\sigma_p(r)$
- **p** is called the **selection predicate (as a filter)**
 - Example: select those tuples of the instructor relation where the instructor is in the “Physics” department.
 - Query

$\sigma_{\text{dept_name}=\text{“Physics”}}(\text{instructor})$

- **Result**

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Select Operation (Cont.)

- We allow **comparisons** using
 $=, \neq, >, \geq, <, \leq$
in the selection predicate.
- We can combine several predicates into a larger predicate by using the connectives:
 $\wedge$ (and), $\vee$ (or), $\neg$ (not)
- Example: Find the instructors in Physics with a salary greater \$90,000, we write:

$$\sigma_{dept_name="Physics" \wedge salary > 90,000} (instructor)$$

- The select predicate may include comparisons between two attributes.
 - Example, find all departments whose name is the same as their building name:

$$\sigma_{dept_name=building} (department)$$

Select Operation Example

$R(a_id, b_id)$

a_id	b_id
a1	101
a2	102
a2	103
a3	104

$\sigma_{a_id='a2'}(R)$

a_id	b_id
a2	102
a2	103

$\sigma_{a_id='a2' \wedge b_id > 102}(R)$

a_id	b_id
a2	103

```
SELECT * FROM R
WHERE a_id='a2' AND b_id>102;
```

$R(a_id, b_id)$

a_id	b_id
a1	101
a2	102
a2	103
a3	104

$\sigma_{a_id='a2'}(R)$

a_id	b_id
a2	102
a2	103

$\sigma_{a_id='a2' \wedge b_id > 102}(R)$

a_id	b_id
a2	103

```
SELECT * FROM R
WHERE a_id='a2' AND b_id>102;
```

Project Operation

- A unary operation that **returns its argument relation**, with certain attributes **left out**.
 - Notation: $\pi_{A_1, A_2, A_3 \dots A_k}(r)$
where $A_1, A_2, \dots, A_k$ are attribute names and r is a relation name.
- The result is defined as the relation of **k columns** obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets

Project Operation Example

- Example: eliminate the *dept_name* attribute of *instructor*

- Query:

$\pi_{ID, name, salary}(instructor)$

- Result:

```
SELECT ID, name, salary
FROM instructor;
```

<i>ID</i>	<i>name</i>	<i>salary</i>
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Composition of Relational Operations

构造

- The result of a relational-algebra operation is relation and therefore of relational-algebra operations can be composed together into a relational-algebra expression.
- Consider the query -- Find the names of all instructors in the Physics department.

$$\pi_{name}(\sigma_{dept_name = "Physics"}(instructor))$$

- Instead of giving the name of a relation as the argument of the projection operation, we give an expression that evaluates to a relation.

Cartesian-Product Operation

- The Cartesian-product operation (denoted by **X**) allows us to **combine information from any two relations**.
 - Example: the Cartesian product of the relations instructor and teaches is written as:
instructor X teaches
- We construct a tuple of the result out of **each possible pair of tuples**: one from the instructor relation and one from the teaches relation (see next slide)
- Since the instructor **ID** appears in both relations we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came.
 - instructor.ID
 - teaches.ID

The *instructor* **X** *teaches* table

<i>instructor.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Join Operation

- The Cartesian-Product

instructor X teaches

associates every tuple of instructor with every tuple of teaches.

- Most of the resulting rows have information about instructors who did NOT teach a particular course.
- To get only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught, we write:

$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$

- We get only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught.
- The result of this expression, shown in the next slide

Join Operation (Cont.)

- The table corresponding to:

$\sigma_{instructor.id = teaches.id}$ (*instructor x teaches*)

<i>instructor.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Join Operation (Cont.)

- The join operation allows us to combine a select operation and a Cartesian-Product operation into a single operation.
- Consider relations $r (R)$ and $s (S)$
- Let “theta” be a predicate on attributes in the schema R “union” S. The join operation $r \bowtie_{\theta} s$ is defined as follows:

$$r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$$

- Thus

$$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$$

- Can equivalently be written as

$$instructor \bowtie_{instructor.id = teaches.id} teaches.$$

Union Operation

- The union operation allows us to **combine two relations**: Notation: $r \cup s$
- For $r \cup s$ to be valid.
 1. r, s must have the **same arity** (same number of attributes)
 2. The attribute domains must be **compatible** (example: 2nd column of r deals with the same type of values as does the 2nd column of s)
- **Example: to find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both**

$\pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017}(section)) \cup$

$\pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018}(section))$

Union Operation (Cont.)

- Result of:

$\pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017}(section)) \cup$

$\pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018}(section))$

<i>course_id</i>

CS-101

CS-315

CS-319

CS-347

FIN-201

HIS-351

MU-199

PHY-101

Set-Intersection Operation

- The set-intersection operation allows us to find tuples that are in both the input relations.
- Notation: $r \cap s$
- Assume:
 - r, s have the *same arity*
 - attributes of r and s are compatible
- Example: Find the set of all courses taught in both the Fall 2017 and the Spring 2018 semesters.

$$\pi_{\text{course_id}}(\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2017}(\text{section})) \cap \pi_{\text{course_id}}(\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2018}(\text{section}))$$

- Result

course_id

CS-101

Set Difference Operation

- The set-difference operation allows us to find tuples that are in one relation but are not in another.
- Notation $r - s$
- Set differences must be taken between **compatible** relations.
 - r and s must have the **same** arity
 - attribute domains of r and s must be compatible
- Example: to find all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester

$\pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017}(section)) -$

$\pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018}(section))$

<i>course_id</i>
CS-347
PHY-101

The **Assignment** Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The **assignment operation** is denoted by $\leftarrow$ and works like assignment in a programming language.
- Example: Find all instructor in the “Physics” and Music department.

Physics $\leftarrow \sigma_{dept_name=“Physics”}(instructor)$

Music $\leftarrow \sigma_{dept_name=“Music”}(instructor)$

Physics $\cup$ *Music*

- With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.

The **Rename** Operation

- The results of relational-algebra expressions do not have a name that we can use to refer to them. The **rename operator**, ρ , is provided for that purpose

- The expression:

$$\rho_x(E)$$

returns the result of expression E under the name x

- Another form of the rename operation:

$$\rho_{x(A1,A2, \dots A_n)}(E)$$

Equivalent Queries

- There is more than one way to write a query in relational algebra.
- **Example:** Find information about courses taught by instructors in the Physics department with salary greater than 90,000

- Query 1

$$\sigma_{dept_name = \text{"Physics"} \wedge salary > 90,000} (instructor)$$

- Query 2

$$\sigma_{dept_name = \text{"Physics"}} (\sigma_{salary > 90,000} (instructor))$$

- The two queries are *not* identical; they are, however, equivalent -- they give the same result on any database.

Equivalent Queries

- There is more than one way to write a query in relational algebra.
- **Example:** Find information about courses taught by instructors in the Physics department
- Query 1

$\sigma_{dept_name = \text{"Physics"}}(instructor \bowtie_{instructor.ID = teaches.ID} teaches)$

- Query 2

$(\sigma_{dept_name = \text{"Physics"}}(instructor)) \bowtie_{instructor.ID = teaches.ID} teaches$

- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.

Introduction to SQL

Outline

- **Overview of The SQL Query Language**
- **SQL Data Definition**
- **Basic Query Structure of SQL Queries**
- **Additional Basic Operations**
- **Set Operations**
- **Null Values**
- **Aggregate Functions**
- **Nested Subqueries**
- **Modification of the Database**

History

- IBM Sequel language developed as part of System R project at the IBM San Jose Research Laboratory
- Renamed **Structured Query Language (SQL)**
- ANSI and ISO standard SQL:
 - SQL-86; SQL-89
 - **SQL-92**
 - SQL:1999 (language name became Y2K compliant!)
 - SQL:2003
 - SQL:2008 → Truncation, Fancy Sorting
 - SQL:2011 → Temporal DBs, Pipelined DML
 - SQL:2016 → JSON, Polymorphic tables
 - SQL:2023 → Property Graph Queries, Muti-Dim. Arrays
- Commercial systems offer most, if not all, **SQL-92** features, plus varying feature sets from later standards and special proprietary features.
 - **Not all examples** here may work on your particular system.

SQL Parts

- **DML** -- provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.
- **Integrity** -- the **DDL** includes commands for specifying *integrity* constraints.
- **View definition** -- The DDL includes commands for **defining views**.
- **Transaction control** -- includes commands for specifying the beginning and ending of transactions.
- **Embedded SQL and dynamic SQL** -- define how SQL statements can be embedded within general-purpose programming languages.
- **Authorization** -- includes commands for specifying access rights to relations and views.

Data Definition Language

- The SQL data-definition language (DDL) allows the specification of information about relations, including:
 - The **schema** for each relation.
 - The **type of values** associated with each attribute.
 - The **Integrity** constraints
 - The **set of indices** to be maintained for each relation.
 - Security and authorization information for each relation.
 - The physical storage structure of each relation on disk.

Domain **Types** in SQL

- **char(*n*)**. Fixed length character string, with user-specified length *n*.
- **varchar(*n*)**. Variable length character strings, with user-specified maximum length *n*.
- **int**. Integer (a finite subset of the integers that is machine-dependent).
- **smallint**. Small integer (a machine-dependent subset of the integer domain type).
- **numeric(*p*,*d*)**. Fixed point number, with user-specified precision of ***p* digits**, with ***d* digits** to the right of decimal **point**. (ex., numeric(3,1), allows 44.5 to be stores exactly, but not 444.5 or 0.32)
- **real, double precision**. Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(*n*)**. Floating point number, with user-specified precision of at least ***n* digits**.
- More are covered later.

DBS Environments

- SQLite3



- Download: <https://www.sqlite.org/2025/sqlite-tools-win-x64-3500400.zip> (6.13 MiB) by 2025/09/07

- MariaDB



- Download: https://mariadb.org/download/?t=mariadb&p=mariadb&r=12.0.2&os=windows&cpu=x86_64&pkg=msi&mirror=blendbyte (81.2 MB)

- PostgreSQL



PostgreSQL

- Download: <https://www.enterprisedb.com/downloads/postgres-postgresql-downloads> (347MB)

SQLite3 Command Line Interface: Tutorial

- A simple command-line program named **sqlite3** (or **sqlite3.exe** on Windows) that allows the user to manually enter and execute SQL statements against an SQLite database or against a ZIP archive.
- Typing "sqlite3" at the command prompt, optionally followed by the name of the file that holds the SQLite database (or ZIP archive).
- If the named file does not exist, a new database file with the given name will be created automatically.
- If no database file is specified on the command-line, a temporary database is created and automatically deleted when the "sqlite3" program exits.

Create Table Construct

- An SQL relation is defined using the **create table** command:

create table *r*

$(A_1 D_1, A_2 D_2, \dots, A_n D_n,$
 (integrity-constraint₁),
 ...,
 (integrity-constraint_k));

- *r* is the name of the relation
 - each A_i is an attribute name in the schema of relation *r*
 - D_i is the data type of values in the domain of attribute A_i
- Example:

```
create table instructor (  
    ID          char(5),  
    name        varchar(20),  
    dept_name    varchar(20),  
    salary       numeric(8,2)  
);
```

Integrity Constraints in Create Table

- Types of integrity constraints
 - primary key ($A_1, \dots, A_n$)
 - foreign key ($A_m, \dots, A_n$) references r
 - not null
- SQL prevents any update to the database that violates an integrity constraint.
- Example:

```
create table instructor (  
    ID          char(5),  
    name        varchar(20) not null,  
    dept_name    varchar(20),  
    salary       numeric(8,2),  
    primary key (ID),  
    foreign key (dept_name) references department  
);
```

And a Few More Relation Definitions

```
create table student (  
    ID          varchar(5),  
    name        varchar(20) not null,  
    dept_name   varchar(20),  
    tot_cred    numeric(3,0),  
    primary key (ID),  
    foreign key (dept_name) references department  
);
```

```
create table takes (  
    ID          varchar(5),  
    course_id   varchar(8),  
    sec_id      varchar(8),  
    semester    varchar(6),  
    year        numeric(4,0),  
    grade       varchar(2),  
    primary key (ID, course_id, sec_id, semester, year),  
    foreign key (ID) references student, foreign key (course_id, sec_id,  
semester, year) references section  
);
```

And more still

```
create table course (  
    course_id    varchar(8),  
    title        varchar(50),  
    dept_name    varchar(20),  
    credits      numeric(2,0),  
    primary key  (course_id),  
    foreign key  (dept_name) references department  
);
```

Updates to tables

- Insert

- `insert into instructor values ('10211', 'Smith', 'Biology', 66000);`

- Delete

- Remove all tuples from the *student* relation

- `delete from student;`

- Drop Table

- `drop table r;`

- Alter

- `alter table r add A D;`

- where *A* is the name of the attribute to be added to relation *r* and *D* is the domain of *A*.
 - All exiting tuples in the relation are assigned *null* as the value for the new attribute.

- `alter table r drop A;`

- where *A* is the name of an attribute of relation *r*
 - Dropping of attributes not supported by many databases.

Basic Query Structure

- A typical SQL query has the form:

```
select A1, A2, ..., An  
from r1, r2, ..., rm  
where P;
```

- A_i represents an **attribute**
- r_i represents a **relation**
- P is a **predicate**.
- The **result** of an SQL query is **a relation**.

The **select** Clause

- The **select** clause lists **the attributes** desired in the result of a query
 - corresponds to the **projection operation** of the relational algebra
- Example: find the names of all instructors:

```
select name  
from instructor;
```

- NOTE: SQL names are **case insensitive** (i.e., you may use upper- or lower-case letters.)
 - E.g., *Name* $\equiv$ *NAME* $\equiv$ *name*
 - Some people use upper case wherever we use bold font.

The select Clause (Cont.)

- SQL allows **duplicates** in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **distinct** after select.
- Find the department names of all instructors, and remove duplicates

```
select distinct dept_name  
from instructor;
```

- The keyword **all** specifies that duplicates should not be removed.

```
select all dept_name  
from instructor;
```

dept_name

Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

The select Clause (Cont.)

- An **asterisk** in the select clause denotes “**all attributes**”

```
select * from instructor;
```

- An attribute can be a literal with **no from** clause

```
select '437';
```

- Results is a table with one column and a single row with value “437”
- Can give the column a name using:

```
select '437' as F00;
```

- An attribute can be a literal with from clause

```
select 'A' as F00  
from instructor;
```

- Result is a table with one column and *N* rows (number of tuples in the *instructors* table), each row with value “A”

The select Clause (Cont.)

- The **select** clause can contain **arithmetic expressions** involving the operation, +, −, *, and /, and operating on constants or attributes of tuples.
 - The query:

```
select ID, name, salary/12  
from instructor;
```

would return a relation that is the same as the *instructor* relation, except that the value of the attribute *salary* is divided by 12.

- Can rename “*salary/12*” using the **as** clause:

```
select ID, name, salary/12 as monthly_salary  
from instructor;
```

The **where** Clause

- The **where** clause specifies **conditions** that the result must satisfy
 - Corresponds to the selection predicate of the relational algebra.
- To find all instructors in Comp. Sci. dept

```
select name  
from instructor  
where dept_name = 'Comp. Sci.';
```

- SQL allows the use of the logical connectives **and**, **or**, and **not**
- The operands of the logical connectives can be expressions involving the comparison operators **<**, **<=**, **>**, **>=**, **=**, and **<>**.
- Comparisons can be applied to results of arithmetic expressions
- To find all instructors in Comp. Sci. dept with salary > 70000

```
select name  
from instructor  
where dept_name = 'Comp. Sci.' and salary > 70000;
```

<i>name</i>
Katz
Brandt

The **from** Clause

- The **from** clause lists the **relations** involved in the query
 - Corresponds to the *Cartesian product* operation of the relational algebra.
- Find the **Cartesian product** *instructor X teaches*

```
select * from instructor, teaches
```

- generates every possible instructor – teaches pair, with all attributes from both relations.
 - For common attributes (e.g., *ID*), the attributes in the resulting table are renamed using the relation name (e.g., *instructor.ID*)
- Cartesian product not very useful directly, but useful combined with where-clause condition (selection operation in relational algebra).

Examples

- Find the names of all instructors who have taught some course and the course_id

```
select name, course_id
from instructor , teaches
where instructor.ID = teaches.ID;
```

- Find the names of all instructors in the Art department who have taught some course and the course_id

```
select name, course_id
from instructor , teaches
where instructor.ID = teaches.ID
      and instructor. dept_name = 'Art';
```

<i>name</i>	<i>course_id</i>
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181

The **Rename** Operation

- The SQL allows renaming relations and attributes using the **as** clause:

old-name **as** *new-name*

- Find the names of all instructors who have a higher salary than some instructor in 'Comp. Sci'.

```
select distinct T.name  
from instructor as T, instructor as S  
where T.salary > S.salary and S.dept_name = 'Comp. Sci.';
```

- Keyword **as** is optional and may be omitted
instructor as T $\equiv$ *instructor T*

Self Join Example

- Relation *emp-super*

<i>person</i>	<i>supervisor</i>
Bob	Alice
Mary	Susan
Alice	David
David	Mary

- Find the supervisor of “Bob”
- Find the supervisor of the supervisor of “Bob”
- Can you find ALL the supervisors (direct and indirect) of “Bob”?

String Operations

- SQL includes a **string-matching operator** for comparisons on character strings. The operator like uses patterns that are described using two special characters:
 - **percent (%)**: matches **any substring**.
 - **underscore (_)**: matches **any character**.
- Find the names of all instructors whose name includes the substring “dar”.

```
select name  
from instructor  
where name like '%dar%'
```

- Match the string “100%”
like '100 \%' escape '\'
in that above we use backslash (\) as the escape character.

String Operations (Cont.)

- Patterns are **case sensitive**.
- Pattern matching examples:
 - 'Intro%' matches any string beginning with “Intro”.
 - '%Comp%' matches any string containing “Comp” as a substring.
 - '___' matches any string of exactly **three** characters.
 - '___%' matches any string of at least three characters.
- SQL supports a variety of string operations such as
 - concatenation (using “||”)
 - converting from upper to lower case (and vice versa)
 - finding string length, extracting substrings, etc.

Ordering the Display of Tuples

- List in alphabetic order the names of all instructors

```
select distinct name  
from instructor  
order by name;
```

- We may specify **desc** for descending order or **asc** for ascending order, for each attribute; ascending order is the default.

- Example: `order by name desc;`

- Can sort on multiple attributes

- Example: `order by dept_name, name;`

Where Clause Predicates

- SQL includes a **between** comparison operator
 - Example: Find the names of all instructors with salary between \$90,000 and \$100,000 (that is, $\geq$ \$90,000 and $\leq$ \$100,000)

```
select name  
from instructor  
where salary between 90000 and 100000;
```

- Tuple comparison

```
select name, course_id  
from instructor, teaches  
where (instructor.ID, dept_name) = (teaches.ID,  
'Biology');
```

Set Operations

- Find courses that ran in Fall 2017 **or in** Spring 2018

```
(select course_id from section where sem = 'Fall' and year = 2017)  
union  
(select course_id from section where sem = 'Spring' and year = 2018)
```

- Find courses that ran in Fall 2017 **and in** Spring 2018

```
(select course_id from section where sem = 'Fall' and year = 2017)  
intersect  
(select course_id from section where sem = 'Spring' and year = 2018)
```

- Find courses that ran in Fall 2017 **but not in** Spring 2018

```
(select course_id from section where sem = 'Fall' and year = 2017)  
except  
(select course_id from section where sem = 'Spring' and year = 2018)
```

Set Operations (Cont.)

- Set operations **union**, **intersect**, and **except**
 - Each of the above operations automatically eliminates duplicates
- To retain all duplicates use the
 - **union all,**
 - **intersect all**
 - **except all.**

Null Values

- It is possible for tuples to have a null value, denoted by null, for some of their attributes
- null signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving null is null
 - Example: $5 + \text{null}$ returns null
- The predicate `is null` can be used to check for null values.
 - Example: Find all instructors whose salary is null.

```
select name  
from instructor  
where salary is null
```

- The predicate `is not null` succeeds if the value on which it is applied is not null.

Null Values (Cont.)

- SQL treats as unknown the result of any comparison involving a null value (other than predicates is null and is not null).
 - Example: $5 < \text{null}$ or $\text{null} <> \text{null}$ or $\text{null} = \text{null}$
- The predicate in a where clause can involve Boolean operations (and, or, not); thus the definitions of the Boolean operations need to be extended to deal with the value unknown.
 - and
 - (true and unknown) = unknown,
 - (false and unknown) = false,
 - (unknown and unknown) = unknown
 - or:
 - (unknown or true) = true,
 - (unknown or false) = unknown
 - (unknown or unknown) = unknown
- Result of where clause predicate is treated as *false* if it evaluates to *unknown*

Aggregate Functions

- These functions operate on the multiset of values of a column of a relation, and return a value

avg: average value

min: minimum value

max: maximum value

sum: sum of values

count: number of values

Aggregate Functions Examples

- Find the average salary of instructors in the Computer Science department

```
select avg (salary)
from instructor
where dept_name= 'Comp. Sci.';
```

- Find the total number of instructors who teach a course in the Spring 2018 semester

```
select count (distinct ID)
from teaches
where semester = 'Spring' and year = 2018;
```

- Find the number of tuples in the *course* relation

```
select count (*) from course;
```

Aggregate Functions – Group By

- Find the average salary of instructors in each department

```
select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name;
```

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Aggregation (Cont.)

- Attributes in select clause outside of aggregate functions must appear in group by list
 - /* erroneous query */

```
select dept_name, ID, avg (salary)
from instructor
group by dept_name, ID;
```

Aggregate Functions – **Having** Clause

- Find the names and average salaries of all departments whose average salary is greater than 42000

```
select dept_name, avg (salary) as avg_salary  
from instructor  
group by dept_name  
having avg (salary) > 42000;
```

- Note: predicates in the **having** clause are applied **after the formation of groups** whereas predicates in the **where** clause are applied **before** forming groups

Nested Subqueries

- SQL provides a mechanism for the nesting of subqueries. A **subquery** is a **select-from-where** expression that is nested within another query.
- The nesting can be done in the following SQL query

```
select A1, A2, ..., An  
from r1, r2, ..., rm  
where P;
```

as follows:

- **From** clause: r_i can be replaced by **any valid subquery**
- **Where** clause: P can be replaced with an expression of the form:
 B <operation> (subquery)
 B is an **attribute** and <operation> to be defined later.
- **Select** clause:
 A_i can be replaced by a subquery that **generates a single value**.

Set Membership

Set Membership

- Find courses offered in Fall 2017 and in Spring 2018

```
select distinct course_id
from section
where semester = 'Fall' and year= 2017 and course_id in
    (select course_id from section
     where semester = 'Spring' and year= 2018);
```

- Find courses offered in Fall 2017 but not in Spring 2018

```
select distinct course_id
from section
where semester = 'Fall' and year= 2017 and course_id not in
    (select course_id from section
     where semester = 'Spring' and year= 2018);
```

Set Membership (Cont.)

- Name all instructors whose name is neither “Mozart” nor Einstein”

```
select distinct name  
from instructor  
where name not in ('Mozart', 'Einstein');
```

- Find the total number of (distinct) students who have taken course sections taught by the instructor with *ID* 10101

```
select count (distinct ID)  
from takes  
where (course_id, sec_id, semester, year) in  
      (select course_id, sec_id, semester, year from teaches  
       where teaches.ID= 10101);
```

- **Note:** Above query can be written in a much simpler manner.
The formulation above is simply to illustrate SQL features

Set Comparison

Set Comparison – “**some**” Clause

- Find names of instructors with salary greater than that of some (at least one) instructor in the Biology department.

```
select distinct T.name  
from instructor as T, instructor as S  
where T.salary > S.salary and S.dept name = 'Biology';
```

- Same query using > some clause

```
select name  
from instructor  
where salary > some (select salary from instructor  
                     where dept_name = 'Biology');
```

Definition of “**some**” Clause

- $F \text{ <comp> some } r \Leftrightarrow \exists t \in r \text{ such that } (F \text{ <comp> } t)$

Where <comp> can be: <, ≤, >, =, ≠

$(5 \text{ < some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{true}$
 (read: 5 < some tuple in the relation)

$(5 \text{ < some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{false}$

$(5 = \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true}$

$(5 \neq \text{some } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true (since } 0 \neq 5)$

(= **some**) ≡ in
 However, (≠ **some**) ≠ not in

Set Comparison – “all” Clause

- Find the names of all instructors whose salary is greater than the salary of all instructors in the Biology department.

```
select name  
from instructor  
where salary > all (select salary from instructor  
                    where dept_name = 'Biology');
```

Definition of “all” Clause

- $F \text{ <comp> all } r \Leftrightarrow \forall t \in r (F \text{ <comp> } t)$

$$(5 < \text{all } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{false}$$

$$(5 < \text{all } \begin{array}{|c|} \hline 6 \\ \hline 10 \\ \hline \end{array}) = \text{true}$$

$$(5 = \text{all } \begin{array}{|c|} \hline 4 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

$$(5 \neq \text{all } \begin{array}{|c|} \hline 4 \\ \hline 6 \\ \hline \end{array}) = \text{true (since } 5 \neq 4 \text{ and } 5 \neq 6)$$

$(\neq \text{ all}) \equiv \text{not in}$
However, $(= \text{ all}) \not\equiv \text{in}$

Test for Empty Relations

- The exists construct returns the value true if the argument subquery is **nonempty**.
- exists $r \Leftrightarrow r \neq \emptyset$
- not exists $r \Leftrightarrow r = \emptyset$

Use of “**exists**” Clause

- Yet another way of specifying the query “Find all courses taught in both the Fall 2017 semester and in the Spring 2018 semester”

```
select course_id
from section as S
where semester = 'Fall' and year = 2017 and
      exists (select *
              from section as T
              where semester = 'Spring' and year = 2018
              and S.course_id = T.course_id);
```

- **Correlation name** – variable S in the outer query
- **Correlated subquery** – the inner query

Use of “**not exists**” Clause

- Find all students who have taken all courses offered in the Biology department.

```
select distinct S.ID, S.name
from student as S
where not exists ( (select course_id from course
                    where dept_name = 'Biology')
                  except
                  (select T.course_id from takes as T
                   where S.ID = T.ID));
```

- First nested query lists all courses offered in Biology
 - Second nested query lists all courses a particular student took
- Note that $X - Y = \emptyset \Leftrightarrow X \subseteq Y$**
- Note: Cannot write this query using = all and its variants**

Test for Absence of Duplicate Tuples

- The **unique** construct tests whether a subquery has any duplicate tuples in its result.
- The **unique** construct evaluates to “true” if a given subquery contains no duplicates .
- Find all courses that were offered at most once in 2017

```
select T.course_id
from course as T
where unique ( select R.course_id
               from section as R
               where T.course_id= R.course_id
                 and R.year = 2017);
```

Subqueries in the From Clause

Subqueries in the From Clause

- SQL allows a subquery expression to be used in the from clause
- Find the average instructors' salaries of those departments where the average salary is greater than \$42,000."

```
select dept_name, avg_salary
from ( select dept_name, avg (salary) as avg_salary
      from instructor
      group by dept_name)
where avg_salary > 42000;
```

- Note that we do not need to use the having clause
- Another way to write above query

```
select dept_name, avg_salary
from ( select dept_name, avg (salary) as avg_salary
      from instructor
      group by dept_name)
      as dept_avg (dept_name, avg_salary)
where avg_salary > 42000;
```

With Clause

- The **with** clause provides a way of defining a temporary relation whose definition is available only to the query in which the **with** clause occurs.
- Find all departments with the maximum budget

```
with max_budget (value) as (select max(budget)
                             from department)
select department.name
from department, max_budget
where department.budget = max_budget.value;
```

Complex Queries using **With** Clause

- Find all departments where the total salary is greater than the average of the total salary at all departments

```
with dept_total (dept_name, value) as
    (select dept_name, sum(salary)
     from instructor
     group by dept_name),
dept_total_avg(value) as
    (select avg(value)
     from dept_total)
select dept_name
from dept_total, dept_total_avg
where dept_total.value > dept_total_avg.value;
```

Scalar Subquery

- Scalar subquery is one which is used where a single value is expected
- List all departments along with the number of instructors in each department

```
select dept_name, ( select count(*)  
                    from instructor  
                    where department.dept_name = instructor.dept_name)  
                as num_instructors  
from department;
```

- Runtime error if subquery returns more than one result tuple

Modification of the Database

- **Deletion** of tuples from a given relation.
- **Insertion** of new tuples into a given relation
- **Updating** of values in some tuples in a given relation

Deletion

- Delete all instructors

```
delete from instructor;
```

- Delete all instructors from the Finance department

```
delete from instructor  
where dept_name= 'Finance';
```

- *Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building.*

```
delete from instructor  
where dept_name in (select dept_name  
                    from department  
                    where building = 'Watson');
```

Deletion (Cont.)

- Delete all instructors whose salary is less than the average salary of instructors

```
delete from instructor
where salary < (select avg (salary)
               from instructor);
```

- Problem: as we delete tuples from *instructor*, the average salary changes
- Solution used in SQL:
 1. First, compute avg (salary) and find all tuples to delete
 2. Next, delete all tuples found above (without recomputing avg or retesting the tuples)

Insertion

- Add a new tuple to *course*

```
insert into course values ('CS-437', 'Database  
Systems', 'Comp. Sci.', 4);
```

- or equivalently

```
insert into course (course_id, title, dept_name, credits)  
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- Add a new tuple to *student* with *tot_creds* set to null

```
insert into student  
values ('3003', 'Green', 'Finance', null);
```

Insertion (Cont.)

- Make each student in the Music department who has earned more than 144 credit hours an instructor in the Music department with a salary of \$18,000.

```
insert into instructor
  select ID, name, dept_name, 18000
  from student
  where dept_name = 'Music' and total_cred > 144;
```

- The select from where statement is evaluated fully before any of its results are inserted into the relation.

Otherwise queries like

```
insert into table1 select * from table1;
```

would cause problem

Updates

- Give a 5% salary raise to all instructors

```
update instructor set salary = salary * 1.05;
```

- Give a 5% salary raise to those instructors who earn less than 70000

```
update instructor  
  set salary = salary * 1.05  
  where salary < 70000;
```

- Give a 5% salary raise to instructors whose salary is less than average

```
update instructor  
  set salary = salary * 1.05  
  where salary < (select avg (salary)  
                  from instructor);
```

Updates (Cont.)

- Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by a 5%
- Write two update statements:

```
update instructor  
  set salary = salary * 1.03  
  where salary > 100000;
```

```
update instructor  
  set salary = salary * 1.05  
  where salary <= 100000;
```

- The order is important
- Can be done better using the case statement (next slide)

Case Statement for Conditional Updates

- Same query as before but with **case statement**

```
update instructor
  set salary =
    case
      when salary <= 100000 then salary * 1.05
      else salary * 1.03
    end
```

Updates with **Scalar Subqueries**

- Recompute and update `tot_creds` value for all students

```
update student S
  set tot_cred = (select sum(credits)
                  from takes, course
                  where takes.course_id = course.course_id
                     and S.ID = takes.ID
                     and takes.grade <> 'F'
                     and takes.grade is not null);
```

- Sets `tot_creds` to null for students who have not taken any course
 - Instead of `sum(credits)`, use:

```
update student S
  set tot_cred =
    case
      when sum(credits) is not null then sum(credits)
      else 0
    end
```

Database System

Thank you! Q/A?

Instructor: Yiwen Shen, Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr